UNIVERSIDAD DEL VALLE DE GUATEMALA

Departamento de Ingeniería en Ciencias de la Computación y Tecnologías de la Información



PROYECTO No. 2

Conversión de gramáticas a CNF y algoritmo CYK

Integrantes

Adriana Martínez – 24086

Mishell Ciprian – 231169

Repositorio de GitHub

Repositorio del proyecto

19 de octubre de 2026

1. Introducción

En este proyecto desarrollamos un programa que permite comprobar si una frase pertenece al lenguaje de una gramática libre de contexto. Para hacerlo, primero convertimos la gramática a Forma Normal de Chomsky (CNF) y después aplicamos el algoritmo Cocke–Younger–Kasami (CYK), con el que podemos determinar si la cadena se puede derivar de esa gramática.

Además de indicar si la entrada se acepta o se rechaza, el programa muestra el tiempo de ejecución. Cuando encuentra una derivación válida, también reconstruye un árbol de análisis para mostrar cómo se forma la frase.

2. Objetivo y diseño de la aplicación

Nuestro objetivo principal fue implementar la simplificación de gramáticas y el algoritmo CYK de manera clara y modular, para que fuera sencillo seguir el funcionamiento del programa. Por eso dividimos el proyecto en funciones con tareas específicas: leer las gramáticas, convertirlas a CNF, reconocer cadenas y manejar la interacción con el usuario.

2.1. Flujo general del programa

El programa sigue este proceso:

- Se lee un archivo TXT y se simplifica su gramática hasta obtener una versión en CNF.
- La frase de entrada se separa en símbolos y se procesa con el algoritmo CYK.
- Se muestra un resultado de SÍ o NO, junto con el tiempo de ejecución y, si la frase se acepta, su árbol de análisis.

Los archivos del proyecto se organizan de la siguiente manera:

Archivo	**Responsabilidad**
`main.py`	Recibe las opciones, prepara las frases y muestra los resultados.
`grammar.py`	Lee la gramática, la simplifica, la convierte a CNF y verifica su consistencia.

Archivo	Responsabilidad
`cyk.py`	Construye la tabla dinámica de CYK y reconstruye el árbol de análisis.
`grammars/english.txt`	Contiene la gramática de inglés utilizada en el enunciado.
`grammars/1.txt` y `grammars/1-cnf.txt`	Contienen la gramática aritmética original y una referencia en CNF.
`tests/test_project.py`	Reúne las pruebas del proyecto.

Los algoritmos principales están implementados mediante funciones. Las clases se utilizan únicamente en las pruebas automáticas, ya que forman parte del formato de `unittest`.

3. Cómo ejecutar el programa

Se necesita Python 3; el programa utiliza solamente la biblioteca estándar. Desde una terminal ubicada en la carpeta del proyecto, se ejecuta:

```
python main.py
```

También es posible probar una entrada específica desde la terminal o activar las opciones de depuración y visualización:

```
python main.py --sentence "she eats a cake with a fork"

python main.py --sentence "she eats" --table

python main.py --grammar grammars/1.txt --sentence "( id * id ) + id" --steps
```

Las opciones disponibles son las siguientes:

Opción	Uso
`--grammar archivo.txt`	Carga una gramática diferente.
`--sentence "entrada"`	Valida una sola cadena y finaliza la ejecución.
`--steps`	Muestra cada etapa de la conversión a CNF.

Opción	Uso
`--table`	Muestra las variables que reconocen cada intervalo de la entrada.

4. Formato de entrada y estructuras

La primera variable que aparece en el archivo se utiliza como símbolo inicial. Las producciones se escriben con una flecha `->`, sus alternativas se separan con `|` y los símbolos del lado derecho deben estar separados por espacios. Por ejemplo:

```
S -> A B
A -> a | e
B -> b
```

El programa considera variables a los símbolos que aparecen a la izquierda de alguna producción. Los demás se interpretan como terminales. Las expresiones `e`, `epsilon` y el símbolo ε representan la cadena vacía y deben aparecer solos en su alternativa.

Internamente, la gramática se almacena en un diccionario con conjuntos de tuplas. Esta estructura permite consultar las reglas de cada variable, evitar producciones duplicadas y representar la cadena vacía mediante una tupla vacía.

Al procesar una frase, el programa conserva los terminales que coinciden exactamente con los de la gramática. Si un símbolo no coincide, lo convierte a minúsculas y elimina los puntos finales. Así se pueden probar las frases del enunciado aunque comiencen con mayúscula o terminen con un punto.

4.1. Gramática de inglés del enunciado

La gramática utilizada para reconocer las frases en inglés es la siguiente:

```
S -> NP VP
VP -> VP PP
VP -> V NP
VP -> cooks | drinks | eats | cuts
PP -> P NP
NP -> Det N
NP -> he | she
```

```
V -> cooks | drinks | eats | cuts
P -> in | with
N -> cat | dog
N -> beer | cake | juice | meat | soup
N -> fork | knife | oven | spoon
Det -> a | the
```

Esta gramática ya cumple con la Forma Normal de Chomsky, porque sus producciones tienen dos variables o un solo terminal en el lado derecho. Aun así, el programa incluye el proceso de simplificación y conversión para trabajar con otras gramáticas que no estén en CNF.

La gramática revisa la estructura de las frases, no su significado. Por eso puede aceptar una oración como `the knife drinks a cat`, aunque semánticamente resulte extraña.

5. Simplificación y conversión a CNF

La Forma Normal de Chomsky permite producciones de la forma $A \to BC$ o $A \to a$. Si la gramática original genera la cadena vacía, también se puede conservar una producción especial $S \to \varepsilon$, siempre que el símbolo inicial no aparezca en el lado derecho de otras reglas.

La función `to_cnf` se encarga de coordinar las transformaciones necesarias para obtener este formato.

5.1. Protección del símbolo inicial

Si el símbolo inicial aparece en el lado derecho de alguna producción, se crea un nuevo símbolo inicial con una regla que apunta al original. Esto permite mantener el tratamiento especial de la cadena vacía y asegurar que el nuevo inicio no aparezca dentro de otras producciones.

5.2. Eliminación de producciones vacías

Primero se identifican las variables anulables, es decir, aquellas que pueden generar ε. Después se agregan alternativas en las que se omiten sus apariciones cuando corresponde.

La producción que genera la cadena vacía se conserva únicamente en el símbolo inicial y solo si esa cadena pertenece al lenguaje original.

5.3. Eliminación de producciones unitarias

Las producciones unitarias tienen la forma $A \to B$. Para eliminarlas, se siguen las conexiones entre variables y se copian las producciones no unitarias que se pueden alcanzar.

Durante este recorrido se utiliza un conjunto de variables visitadas para evitar ciclos.

5.4. Eliminación de símbolos inútiles

En esta etapa se conservan únicamente las variables productivas y alcanzables desde el símbolo inicial. De esta forma se eliminan las reglas que no contribuyen a generar cadenas válidas del lenguaje que se quiere reconocer.

5.5. Separación de terminales y división de reglas largas

Cuando un terminal aparece en una producción con más de un símbolo en el lado derecho, se sustituye por una variable auxiliar.

Después, las reglas largas se dividen en pares hasta que cumplen con el formato binario de CNF. Al terminar, se verifica que todas las producciones respeten la forma normal.

6. CYK y programación dinámica

El algoritmo CYK utiliza programación dinámica para determinar si una cadena pertenece al lenguaje de una gramática en CNF. Primero resuelve los fragmentos más pequeños de la entrada y luego aprovecha esos resultados para analizar fragmentos más grandes.

Cada celda de la tabla de CYK guarda las variables capaces de generar un intervalo específico de la cadena.

El proceso comienza llenando la diagonal con las variables que producen cada terminal individual. Luego se analizan los fragmentos de longitud 2, después los de longitud 3 y así sucesivamente.

Para cada intervalo se prueban todas las divisiones posibles. Si una variable B genera la parte izquierda, una variable C genera la parte derecha y existe una producción $A \to BC$, entonces A se agrega a la celda que corresponde al fragmento completo.

Por ejemplo, para la entrada `she eats`, se obtiene lo siguiente:

```
[1, 1] she      => {NP}
[2, 2] eats     => {V, VP}
[1, 2] she eats => {S}
```

Como existe una producción de la forma S → NP VP, el símbolo inicial aparece en el intervalo que contiene toda la frase y la entrada se acepta.

En el código, las posiciones comienzan desde cero. Sin embargo, al mostrar la tabla se presentan desde uno para que sea más fácil leerla.

Con una gramática fija, CYK tiene una complejidad temporal de $O(n^3)$ y una complejidad espacial de $O(n^2)$, donde n es la cantidad de símbolos de la entrada. La conversión a CNF representa un costo adicional, que puede aumentar cuando la gramática tiene muchas variables anulables.

7. Árbol de análisis y medición del tiempo

Además de registrar que una variable reconoce un intervalo, cada entrada de la tabla puede guardar la información necesaria para reconstruir cómo se obtuvo ese resultado: el punto de división y las variables hijas.

Esto permite recuperar un árbol de análisis cuando la frase es aceptada, sin tener que repetir la búsqueda. Si la gramática es ambigua, esta implementación conserva el primer árbol que encuentra.

Para la frase `she eats`, el árbol se representa así:

```
S
    NP
        she
    VP
        eats
```

El tiempo de ejecución se mide con `perf_counter` y se presenta en milisegundos. La medición incluye la creación de los índices, el llenado de la tabla y la reconstrucción del árbol. No incluye la lectura del archivo, la espera de la frase de entrada ni la impresión de los resultados.

Los tiempos observados en las capturas se comentan en la sección de pruebas. Cada valor corresponde a una ejecución individual y puede cambiar al volver a ejecutar el programa.

7.1. Ejemplo de entrada y resultado

Para validar una frase corta, se puede utilizar el siguiente comando:

```
python main.py --sentence "she eats"
```

El siguiente resumen reúne la entrada, el resultado, el tiempo medido y el árbol de análisis. Se presenta como ejemplo explicativo; el formato exacto de los mensajes en la terminal puede ser diferente.

```
Entrada: she eats
Resultado: SI
Tiempo CYK: <valor medido> ms

S
    NP
        she
    VP
        eats
```

La frase se acepta porque `she` puede derivarse de NP, `eats` puede derivarse de VP y la regla S → NP VP permite combinar ambas partes.

Para una frase rechazada, como `she cake eats`, el resultado es NO porque el orden de las palabras no puede derivarse mediante las reglas de la gramática. En ese caso no existe un árbol de análisis válido para la entrada completa.

8. Ejemplos y pruebas realizadas

Probamos frases aceptadas y rechazadas para revisar el orden de las palabras y el vocabulario permitido por la gramática. La siguiente tabla presenta algunos de los casos más representativos:

Entrada	**Resultado**	**Qué revisamos**
`she eats a cake with a fork`	Sí	Objeto y frase preposicional.
`the cat drinks the beer`	Sí	Sujeto y objeto con determinante.
`he cooks`	Sí	Verbo sin objeto.

Entrada	Resultado	Qué revisamos
`she eats the soup in the oven`	Sí	Frase preposicional después del objeto.
`the knife drinks a cat`	Sí	Sintaxis válida, aunque el significado sea extraño.
`she cake eats`	No	Orden incompatible con las reglas.
`the cat drinks beer`	No	Falta el determinante del objeto.
`she eats a pizza`	No	La palabra `pizza` no pertenece a la gramática.
`she`	No	Falta el predicado.
Cadena vacía con `english.txt`	No	La gramática no genera ε.
`( id * id ) + id` con `1.txt`	Sí	Expresión aritmética válida.
`id +` con `1.txt`	No	Falta el operando derecho.

8.1. Resultados de la ejecución manual

Primero probamos el programa en modo interactivo con las frases `he cooks`, `she eats a cake with a fork` y `the cat drinks the beer`. Las tres fueron aceptadas. En el árbol de la segunda frase se observa cómo se combinan el sujeto, el verbo, el objeto y la frase preposicional `with a fork`.

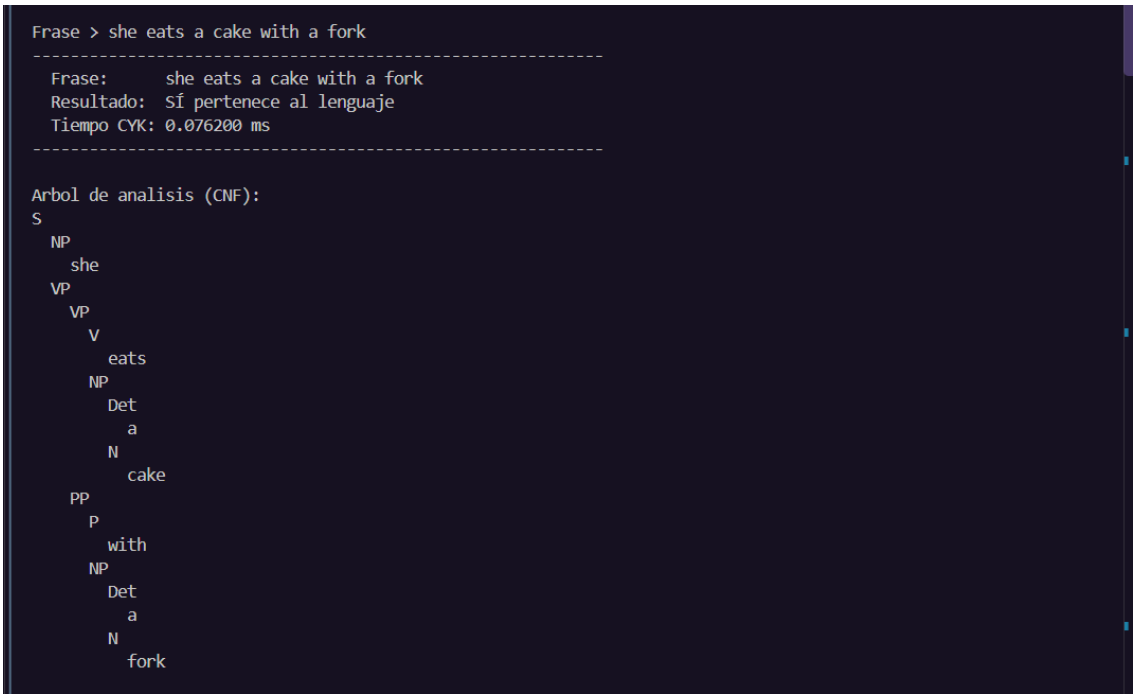


Figura 1: Resultado y árbol de análisis de una frase aceptada.

Después revisamos tres entradas incorrectas por motivos distintos. `she cake eats` altera el orden de las palabras; `the cat drinks beer` omite el determinante del objeto; y `she eats a pizza` incluye una palabra que no pertenece a la gramática. Las tres fueron rechazadas y no se generó un árbol de análisis.

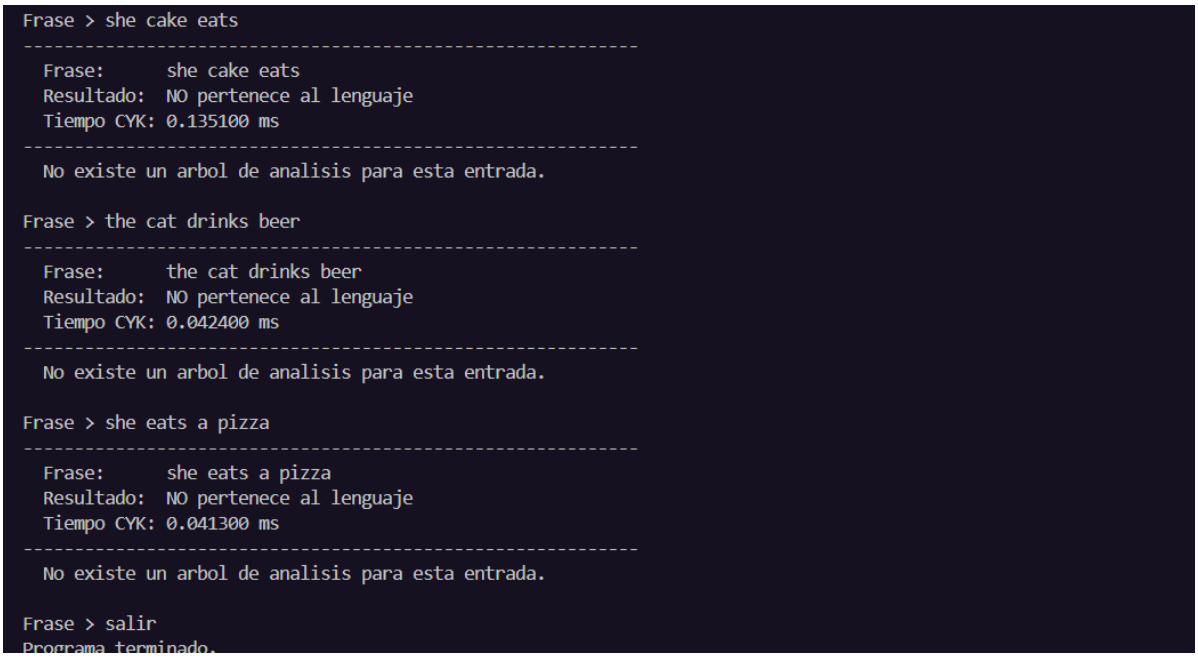


Figura 2: Rechazo por orden incorrecto, falta de determinante y vocabulario no admitido.

Para seguir el proceso de CYK utilizamos la opción `-table` con `she eats a cake with a fork`. La tabla reconoce `a cake` y `a fork` como `NP`, y `with a fork` como `PP`. El intervalo final `[1, 7]` contiene el símbolo inicial `S`, que indica que la frase completa pertenece al lenguaje. Las celdas vacías corresponden a fragmentos que ninguna variable puede generar.

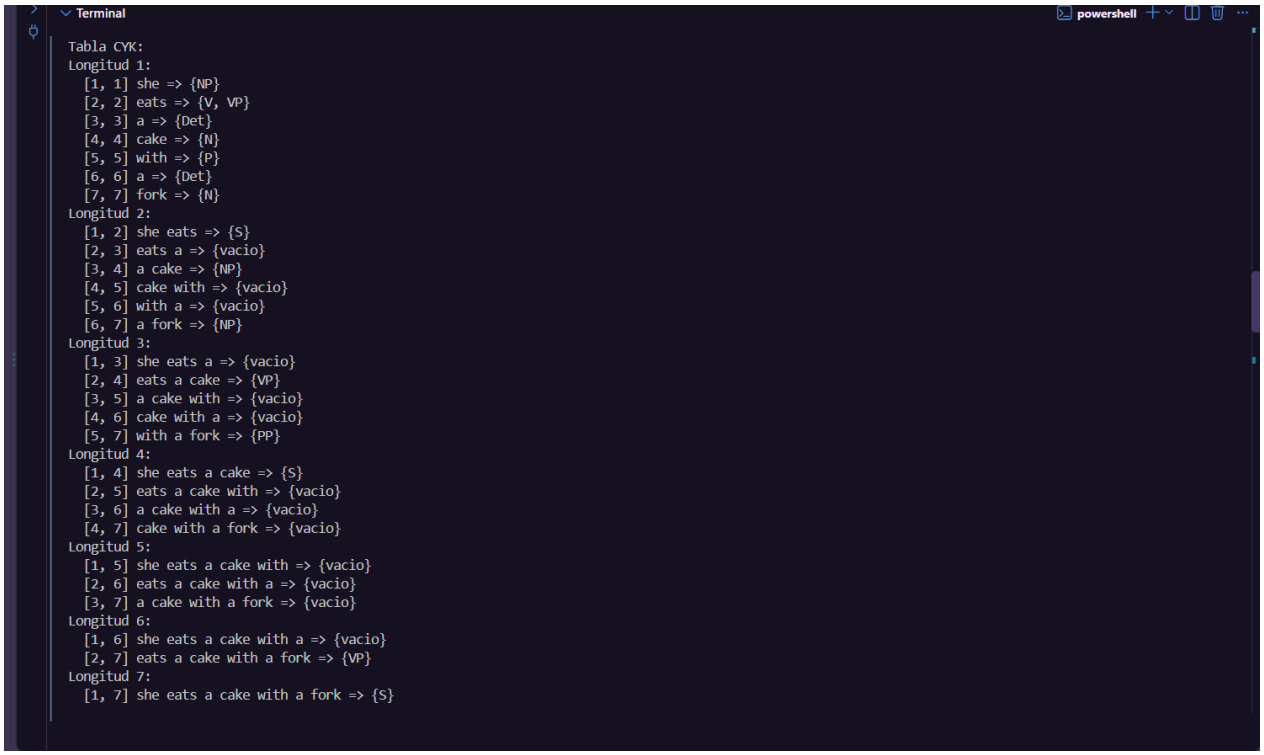


Figura 3: Tabla CYK: el intervalo completo contiene el símbolo inicial.

También cargamos la gramática aritmética `grammars/1.txt` y probamos `id +`. Los símbolos se reconocen de manera individual, pero la celda del intervalo completo queda vacía porque falta el operando derecho. El programa rechaza la entrada e indica que no existe un árbol de análisis.

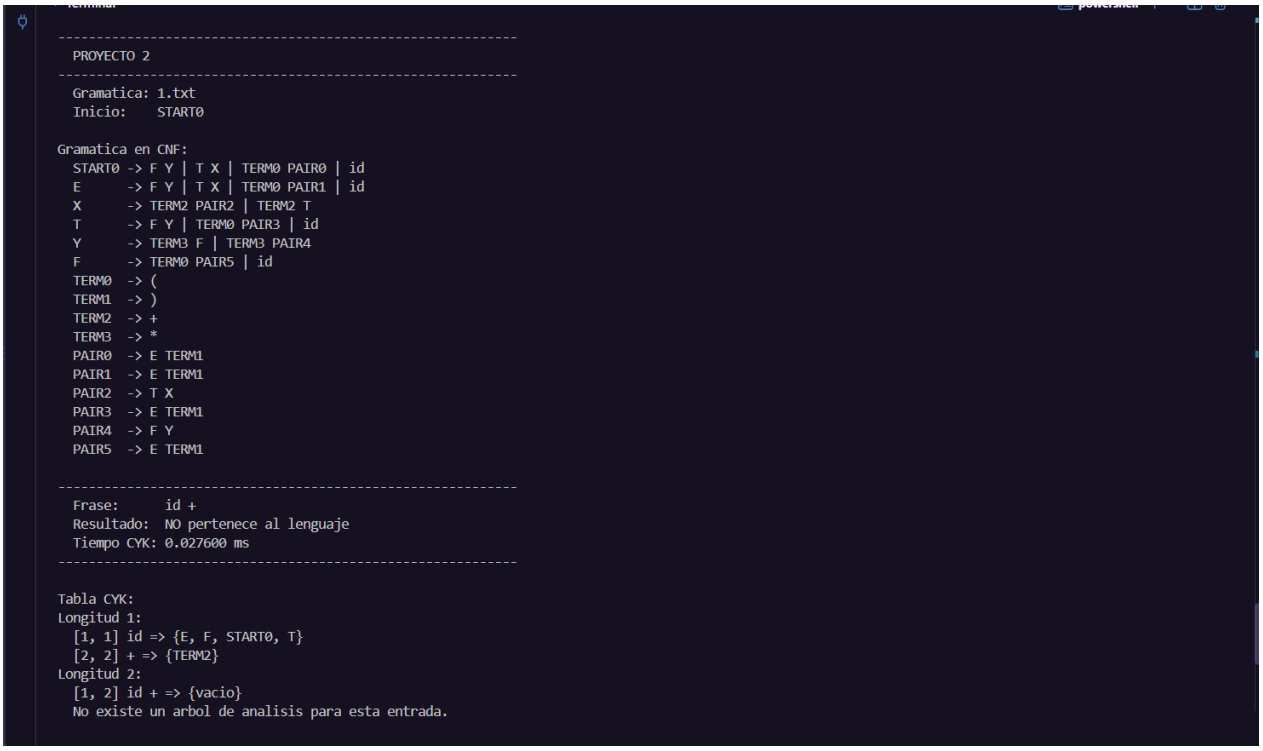


Figura 4: Gramática aritmética convertida a CNF y rechazo de una expresión incompleta.

Los tiempos mostrados son mediciones individuales. Por ejemplo, `she eats a cake with a fork` tomó 0.076200 ms en modo interactivo y 0.059500 ms en la ejecución con tabla. Estos valores pueden variar entre ejecuciones y no representan promedios.

8.2. Pruebas automáticas

Ejecutamos la suite con el siguiente comando:

```
python -m unittest discover -s tests -v
```

La salida mostró `Ran 15 tests` y `OK`. Las pruebas incluyen frases aceptadas y rechazadas, ciclos unitarios, variables anulables repetidas, símbolos inútiles, reglas largas, ambigüedad y errores de formato. En las frases aceptadas también se comprueba que las hojas del árbol coincidan con las palabras de entrada.

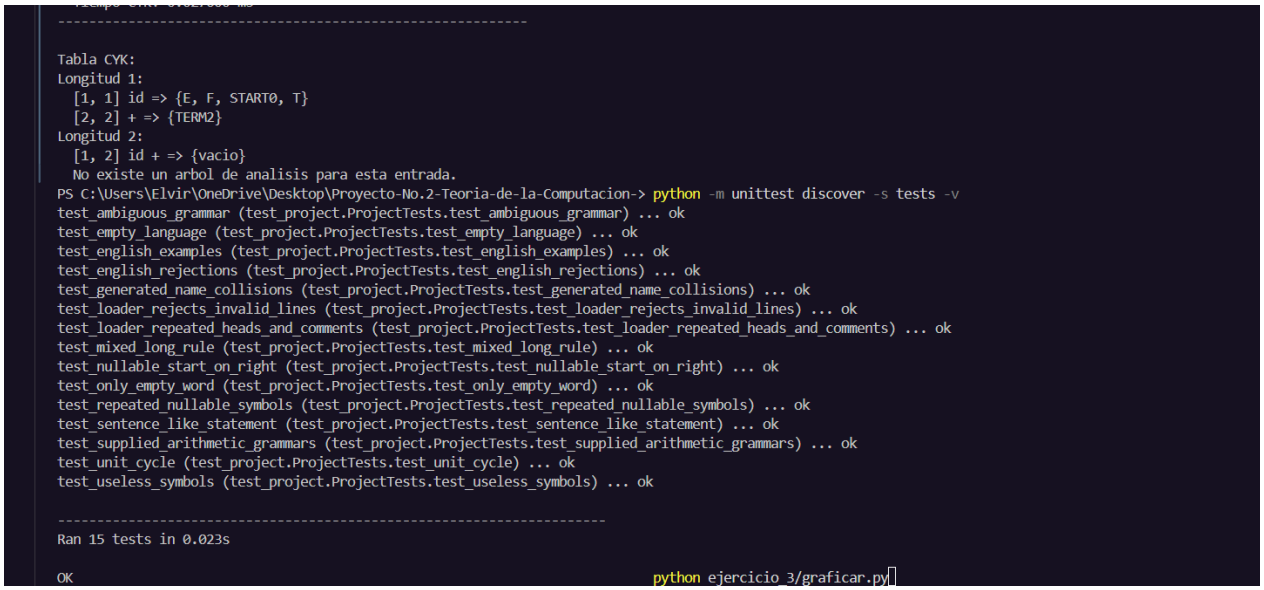


Figura 5: Ejecución de las 15 pruebas automáticas sin fallos.

Además, las pruebas comparan palabras cortas generadas por la gramática original y su versión en CNF. Para las dos gramáticas aritméticas se comparan los lenguajes hasta cinco símbolos. Los resultados coinciden dentro de esos límites; esta comprobación no demuestra equivalencia para todas las cadenas posibles.

9. Discusión, alcances y mejoras

Una de las partes más delicadas fue distinguir correctamente entre variables y terminales durante la limpieza de la gramática. Si la clave de una variable se elimina demasiado pronto, ese símbolo podría interpretarse después como un terminal. Para evitarlo, mantuvimos las claves mientras se evaluaban la productividad y el alcance.

También fue necesario prestar atención a las variables anulables. En una producción como $S \to A\ A\ B$, cada aparición de A debe poder omitirse de manera independiente. El uso de conjuntos permitió reunir todas las alternativas sin duplicar producciones equivalentes.

Como mejora futura, se podrían guardar varias derivaciones por variable para mostrar todos los árboles posibles cuando una gramática sea ambigua. Otra opción sería incorporar un tokenizador más robusto, que permita ingresar expresiones aritméticas sin tener que colocar espacios entre los operadores y los paréntesis.

Además, sería útil estudiar con mayor detalle cuánto crece la conversión a CNF cuando se trabaja con gramáticas grandes o con muchas variables anulables.

10. Conclusión

Este proyecto nos permitió relacionar la simplificación de gramáticas con un algoritmo clásico de reconocimiento. La conversión a CNF prepara la gramática para que CYK pueda procesarla de forma ordenada, mientras que la tabla de programación dinámica reutiliza los resultados parciales para decidir si una cadena pertenece al lenguaje.

El árbol de análisis y las etapas de conversión también ayudan a comprender el proceso y a explicar por qué una entrada se acepta o se rechaza.

11. Referencias

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed., Pearson, 2006.
- Sipser, M. *Introduction to the Theory of Computation*. 3rd ed., Cengage Learning, 2012.
- McDermott, D. Yale University. *CYK Algorithm*. Notas de clase, 12 de noviembre de 2014. `https://zoo.cs.yale.edu/classes/cs470/2014--15-fall/14f-lecnotes/lec31.html`

- University of Washington. *Normal Forms for Context-Free Grammars*. https://courses.cs.washington.edu/courses/cse322/05au/normal.pdf
- University of Illinois Urbana-Champaign. *Context-free languages and grammars*. https://courses.grainger.illinois.edu/cs374/fa2019/A/scribbles/ContextFreeSlides_PublishedVersion.pdf
- Université de Pennsylvanie / CIS. *Context-Free Grammars, Context-Free Languages, and Pushdown Automata*. https://www.cis.upenn.edu/~jean/gbooks/tcbookpdf2.pdf